

```
def getTitle(self):
    return self.title

def payPerYear(self):
    # 52 weeks * 5 days a week * 8 hours per day
    pay = 52 * 5 * 8 * self.ratePerHour
    return pay
```

---

*Listing 10-1: The `Employee` class, which will be used as a base class*

The `Employee` class has the methods `__init__()`, `getName()`, `getTitle()`, and `payPerYear()`. It also has three instance variables, `self.name`, `self.title`, and `self.ratePerHour`, which are set in the `__init__()` method. We retrieve the name and title using getter methods. These employees are paid per hour, so `self.payPerYear()` does a calculation to determine the annual pay based on the hourly rate. Everything in this class should be familiar to you; there is nothing new here. You can instantiate an `Employee` object by itself, and it will work fine.

## ***Subclass: Manager***

For the `Manager` class, we consider the differences between a manager and an employee: the manager is a salaried employee who has a number of direct reports. If this manager does a good job, they get a 10 percent bonus for the year. The `Manager` class can extend the `Employee` class, since the manager is an employee but has additional capabilities and responsibilities.

[Listing 10-2](#) shows the code of our `Manager` class. It only needs to contain code that is different from the `Employee` class, so you'll see that it doesn't have a `getName()` or `getTitle()` method. Any calls to those methods with a `Manager` object will be handled by the methods in the `Employee` class.

## **File:**

### **EmployeeManagerInheritance/EmployeeManagerInheritance.py**

---

```
# Define a Manager subclass that inherits from Employee
```

```
❶ class Manager(Employee):
    def __init__(self, name, title, salary, reportsList=None):
        ❷ self.salary = float(salary)
```